`MixedComplementarityProblems.jl`: A Fast, Batched, Open-Source Interior-Point Solver for Mixed Complementarity Problems

David Fridovich-Keil
Department of Aerospace Engineering and Engineering Mechanics
The University of Texas at Austin
Austin, TX, USA
dfk@utexas.edu

***Abstract*—Mixed complementarity problems (MCPs) arise as the first-order optimality conditions of nonlinear programs and noncooperative games, and give a natural formulation for the multi-agent trajectory optimization problems that appear throughout robotics. The dominant solver for problems of this form is `PATH`, a mature but closed-source package. We present `MixedComplementarityProblems.jl`, an open-source, pure Julia implementation of an interior point method for parametric MCPs that (i) matches `PATH`'s reliability on standard benchmarks and (ii) natively supports batched solving, in which many parameter instances are solved simultaneously and in parallel, either across CPU threads or on an NVIDIA GPU, behind a single solver implementation. On a multi-agent lane-change trajectory game representative of robotics planning problems, our CPU-multithreaded batched solver clears a batch of parametric instances about two orders of magnitude faster than sequential calls to `PATH`. A GPU backend, running the same solver implementation unmodified, also clears these batches far faster than `PATH`, but does not outperform the multithreaded CPU on this problem; the GPU pulls ahead only once each per-instance KKT system grows large, and we characterize this regime dependence. We describe the solver's interior point formulation, the abstraction that lets a single solver implementation run unmodified across dense, batched-sparse, and single-large linear-algebra backends, and report benchmarks against `PATH` on both randomly generated quadratic programs and trajectory games.**

I. Introduction

Multi-agent trajectory planning problems in robotics, ranging from autonomous driving to human-robot interaction, are naturally posed as noncooperative dynamic games, in which each agent optimizes its own trajectory subject to the others' choices. The first-order necessary conditions of optimality for each agent in the game take the form of a mixed complementarity problem (MCP). Solving MCPs rapidly, therefore, becomes a core computational burden for real-time decision-making in multi-agent robotics applications.

The dominant solver for this class of problems is `PATH` [1]. `PATH` is mature, reliable, and widely used; but, it is closed-source, solves one problem instance at a time, and was not designed to support automatic differentiation of solutions with respect to problem parameters. All three pose serious, practical limitations for use in robotics: (i) developers must have access to solver internals in order to customize to the problem at hand, (ii) scenario- and contingency-based planning (e.g., [2], [3]) requires solving a *batch* of identically-structured but differently *parameterized* problems, and (iii) modern machine learning pipelines often embed optimization problem solvers as "layers" [4], and end-to-end differentiability requires us to be able to differentiate a solver's output with respect to its input parameters.

We present `MixedComplementarityProblems.jl`, an open-source, pure Julia solver for parametric MCPs that addresses all of these practical challenges. Concretely, we contribute:

- A simple and easily-customized interior point method for parametric MCPs that matches `PATH`'s reliability on standard benchmarks, while supporting automatic differentiation of solutions with respect to problem parameters.
- A native *batched* solve mode, in which many parameter instances sharing one MCP structure are solved simultaneously, parallelized across CPU threads or an NVIDIA GPU behind a single solver implementation.
- An abstraction boundary defined by a small verb set that separates the interior point loop and its linear algebra backend, which lets a single solver implementation run unmodified across dense, batched-sparse, and single-large-instance regimes, so that supporting a new device or linear-algebra strategy never requires touching the solver loop itself.

As Figure 1 previews and Section V quantifies, on a multi-agent lane-change trajectory game representative of robotics planning problems, clearing a batch of parametric instances with our batched solver is about two orders of magnitude faster than clearing the same batch with sequential calls to `PATH` (Section V-C).

II. Background: Mixed Complementarity Problems

A. Problem definition

A mixed complementarity problem is specified by a map $F : \mathbb{R}^n \to \mathbb{R}^n$ and bounds $\ell, u \in (\mathbb{R} \cup \{\pm\infty\})^n$; a solution is a vector $z \in \mathbb{R}^n$ satisfying, for every coordinate dimension i,

$$\begin{aligned} \ell_i < z_i < u_i &\implies F_i(z) = 0, \\ z_i = \ell_i &\implies F_i(z) \geq 0, \\ z_i = u_i &\implies F_i(z) \leq 0. \end{aligned} \tag{1}$$

Readers are referred to [5] for a complete introduction to MCPs and related problems.

[TODO: replace with rendered lane-change trajectory game schematic, showing the two vehicles' solved trajectories overlaid with a faint fan of trajectories from other initial conditions in the same batch]

(a) One multi-agent trajectory game is one instance out of a batch of parametrically related MCPs, solved simultaneously.

[TODO: replace with wall-clock-vs-batch-size plot: `PATH` (serial), CPU-batched, and GPU-batched, log-scaled y-axis]

(b) Clearing that batch: sequential `PATH` calls versus our batched CPU and GPU solves.

Fig. 1: **[TODO: finalize caption once the real figure is in]** Solving a batch of parametric trajectory-game instances (a) with `MixedComplementarityProblems.jl`'s batched interior point solver is substantially faster than solving the same batch with sequential calls to `PATH` (b).

B. MCPs from KKT conditions

Consider a parametric nonlinear program

$$\min_{x} \; f(x;\theta) \quad \text{s.t.} \quad g(x;\theta) = 0,\; h(x;\theta) \geq 0, \tag{2}$$

with primal variables $x \in \mathbb{R}^{n_x}$, parameters θ, and Lagrange multipliers $\lambda \in \mathbb{R}^{n_\lambda}$ associated with the equality constraint g and $\mu \in \mathbb{R}^{n_\mu}_{\geq 0}$ associated with the inequality constraint h. The first order necessary (i.e. Karush-Kuhn-Tucker, KKT) conditions of optimality for problem (2) are

$$0 = \overbrace{\begin{bmatrix} \nabla_x f(x;\theta) - \nabla_x g(x;\theta)^\top \lambda - \nabla_x h(x;\theta)^\top \mu \\ g(x;\theta) \end{bmatrix}}^{G(x,\lambda,\mu;\theta)} \tag{3a}$$

$$0 \leq \underbrace{h(x;\theta)}_{H(x;\theta)} \perp \mu \geq 0. \tag{3b}$$

Problem (3) can be converted into the form (1) by taking $z = (x, \lambda, \mu)$, $F = (G, H)$, $u_i = \infty$ $(\forall i)$, $\ell_i = -\infty$ $(\forall i \in \{1, 2, \ldots, n_x + n_\lambda\})$, and $\ell_i = 0$ $(\forall i > n_x + n_\lambda)$. Because of the natural connection between KKT conditions (e.g., arising from trajectory optimization problems in robotics) and this *primal-dual* form of the MCP, it is the only form `MixedComplementarityProblems.jl` exposes.

C. MCPs from noncooperative games

The construction in Section II-B extends from one decision maker to many. Consider N agents, where agent i solves

$$\forall i \begin{cases} \min_{x_i} \; f_i(x_i, x_{\neg i};\theta) \\ \text{s.t.} \quad g_i(x_i, x_{\neg i};\theta) = 0,\; h_i(x_i, x_{\neg i};\theta) \geq 0 \end{cases} \tag{4a}$$

$$\text{s.t.} \quad g_{\text{sh}}(x;\theta) = 0,\; h_{\text{sh}}(x;\theta) \geq 0, \tag{4b}$$

subject to its own equality and inequality constraints (g_i and h_i, respectively) and to constraints $g_{\text{sh}}, h_{\text{sh}}$ shared across agents (e.g., pairwise collision avoidance). Each agent i's decision variable is $x_i \in \mathbb{R}^{n_{x_i}}$, and $x_{\neg i}$ denotes the other agents' decisions so that $x = (x_i, x_{\neg i}) = (x_1, \ldots, x_N)$. Writing out each agent's KKT conditions as in (3) and concatenating them—with the shared constraints $g_{\text{sh}}, h_{\text{sh}}$ coupling agents through shared multipliers $\lambda_{\text{sh}}, \mu_{\text{sh}}$—yields a single primal-dual MCP whose solution is a generalized Nash equilibrium: a strategy profile from which no agent can unilaterally improve its own objective [5].

Concretely, concatenate the Lagrange multipliers corresponding to equality constraints as $\lambda := (\lambda_1, \ldots, \lambda_N, \lambda_{\text{sh}})$, and those for inequality constraints as $\mu := (\mu_1, \ldots, \mu_N, \mu_{\text{sh}})$. Then, private multipliers as $\lambda_{\text{pr}} = (\lambda_1, \ldots, \lambda_N)$, $\mu_{\text{priv}} = (\mu_1, \ldots, \mu_N)$. Each agent i's Lagrangian is given by

$$\begin{aligned} \mathcal{L}_i(x, \lambda, \mu;\theta) := f_i(x;\theta) - \lambda_i^\top g_i(x;\theta) - \mu_i^\top h_i(x;\theta) - \\ \lambda_{\text{sh}}^\top g_{\text{sh}}(x;\theta) - \mu_{\text{sh}}^\top h_{\text{sh}}(x;\theta). \end{aligned} \tag{5}$$

The N agents' concatenated KKT conditions are then an instance of (3), with

$$\begin{aligned} G &:= (\nabla_{x_1} \mathcal{L}_1, \ldots, \nabla_{x_N} \mathcal{L}_N, g_1, \ldots, g_N, g_{\text{sh}}), \\ H &:= (h_1, \ldots, h_N, h_{\text{sh}}), \end{aligned} \tag{6}$$

with G and H functions of (x, λ, μ) and parameterized by θ.

Running example: A lane-change trajectory game. Consider the two-agent lane changing trajectory game of Figure 1(a). Here, each agent's decision variable $x_i = \{s_i(t), a_i(t)\}_{t=1}^T$ consists of a state-action *trajectory* over time horizon T. The private equality constraints g_i encode system dynamics $\big(s_i(t), a_i(t)\big) \to s_i(t+1)$, private inequality constraints encode actuator limits $\|a_i(t)\| \leq \bar{a}_i$, and shared inequality constraints encode collision-

avoidance $\|P(s_1(t) - s_2(t))\| \geq \Delta$ (with projection matrix P recovering agents' position). Agents' objective functions f_i encode lane and speed preferences, and penalize control effort. Parameters θ encode the agents' initial states $s_i(1)$, lane preferences, desired speeds, actuator limits $\bar{a}_i$, and collision-avoidance radius Δ.

Observe that the shared collision avoidance constraint—and any nonlinearity in agents' dynamics—renders the feasible set nonconvex.

III. Solver Design

`MixedComplementarityProblems.jl` implements an intentionally-simple primal-dual interior point method to solve problem (3). To that end, we introduce a slack variable $z \in \mathbb{R}^{n_\mu}$ and rewrite (3) in terms of $z \geq 0$ and a homotopy parameter $\rho > 0$:

$$K_\rho(x, \lambda, \mu, z; \theta) := \begin{bmatrix} G(x, \lambda, \mu; \theta) \\ H(x; \theta) - z \\ z \odot \mu - \rho 1 \end{bmatrix} = 0. \tag{7}$$

Note that the last row of K_ρ involves the elementwise product $z \odot \mu$ and, consequently, the number of equations in K_ρ is equal to the total dimension of the primal, dual, and slack variables (x, λ, μ, z).

As summarized in Algorithm 1, we proceed in rounds by solving (7) via a regularized Newton method and decaying $\rho \to 0$ until reaching the user's defined tolerance $\|K_0\| \leq \bar{K}$. For a given value of ρ, the regularized Newton direction $\delta := (\delta_x, \delta_\lambda, \delta_\mu, \delta_z)$ solves

$$\left(\begin{bmatrix} \nabla_x G & \nabla_\lambda G & \nabla_\mu G & 0 \\ \nabla_x H & 0 & 0 & -I \\ 0 & 0 & \mathrm{dg}(z) & \mathrm{dg}(\mu) \end{bmatrix} + \epsilon I \right) \delta = -K_\rho, \tag{8}$$

with $\epsilon \geq 0$ a regularization parameter chosen as described below, and $\mathrm{dg}(\cdot)$ returning a square matrix with its argument on the main diagonal and all other entries zero.

We implement two additional subtleties. First, on line 5, we conduct a "fraction to the boundary" linesearch to control the rate at which $z, \mu \geq 0$ can approach 0 (cf. [6, §19]). At each iteration k, stepsizes α_k, β_k are chosen so that

$$\begin{aligned} \alpha_k &= \max\left(\alpha \in [0,1] \;:\; z + \alpha \delta_z \geq (1-\tau) z\right) \\ \beta_k &= \max\left(\beta \in [0,1] \;:\; \mu + \beta \delta_\mu \geq (1-\tau) \mu\right), \end{aligned} \tag{9}$$

where the inequalities are interpreted elementwise and $\tau \in (0,1)$ controls the rate of decay. Linesearch (9) has a closed-form, exact solution, which makes it amenable to batched computation as described in Section IV.

Second, we construct an *adaptive* choice of the Newton step regularization parameter ϵ and the interior point homotopy parameter ρ that aims to control subproblem conditioning. At the end of each inner loop (lines 10 and 12), if the solver successfully drove $\|K_\rho\| \leq \rho$ then we *tighten* ρ and ϵ at a rate dependent upon the number of Newton steps required to converge; conversely, if the inner loop could not meet the tolerance and $\|K_\rho\| > \rho$, we *loosen* both ρ and ϵ to improve the conditioning of the next subproblem and require a less precise solution.

Algorithm 1: Primal-dual interior point method for (3)

Input: initial iterate (x, λ, μ, z) with $\mu, z > 0$;
parameters θ; initial $\rho, \epsilon > 0$;
tightening/loosening rates $\underline{\gamma}, \bar{\gamma} \in (0,1)$; max. inner iterations $k_{\max}$; tolerance $\bar{K}$

Output: (x, λ, μ, z) with $\|K_0(x, \lambda, \mu, z; \theta)\| \leq \bar{K}$

1 **while** $\|K_0(x, \lambda, \mu, z; \theta)\| > \bar{K}$ **do**
2 $k \leftarrow 0$
3 **repeat**
4 assemble K_ρ (7) and solve (8) for the Newton direction $\delta = (\delta_x, \delta_\lambda, \delta_\mu, \delta_z)$
5 compute step sizes α_k, β_k via the fraction-to-boundary rule (9)
6 $x \leftarrow x + \alpha_k \delta_x, \quad \lambda \leftarrow \lambda + \alpha_k \delta_\lambda,$ $z \leftarrow z + \alpha_k \delta_z, \quad \mu \leftarrow \mu + \beta_k \delta_\mu$
7 $k \leftarrow k + 1$
8 **until** $\|K_\rho(x, \lambda, \mu, z; \theta)\| \leq \rho$ *or* $k \geq k_{\max}$
9 **if** $\|K_\rho(x, \lambda, \mu, z; \theta)\| \leq \rho$ **then**
10 $\rho \leftarrow \rho\,(1 - e^{-\underline{\gamma} k}), \quad \epsilon \leftarrow \epsilon\,(1 - e^{-\underline{\gamma} k})$ `// tighten`
11 **else**
12 $\rho \leftarrow \rho\,(1 + e^{-\bar{\gamma} k}), \quad \epsilon \leftarrow \epsilon\,(1 + e^{-\bar{\gamma} k})$ `// loosen`
13 $\rho \leftarrow \min(\rho, 1)$
14 **return** (x, λ, μ, z)

IV. Batched and GPU-Parallel Solving

[TODO: notation cleanup before shipping: (i) subscripts on primal/dual variables mean "player i" in Section II-C but "batch instance b" here—pick distinct symbols, or add an explicit disambiguating remark; (ii) batched primal/dual iterates are capitalized $(X, \Lambda, M, Z, \Theta)$ but the batched stepsizes $\alpha_k, \beta_k \in \mathbb{R}^B$ are not—make the capitalization convention consistent across both]

Many applications require solving not one but a whole *batch* of MCPs that share the same symbolic structure (G, H) and differ only in their parameter vector θ—for example, if we wished to solve the trajectory game of Figure 1 from many initial conditions. Rather than naively deploying Algorithm 1 on each instance in series, we reuse the shared symbolic structure and exploit hardware-level parallelism (both CPU multithreading and GPU) to solve an entire batch of B parametrically-related instances in a single call.

Running example: A batch of lane-change games. Sampling B initial conditions, lane preferences, or collision radii of Figure 1(a) yields B instances of the same symbolic game, differing only in θ; stacking them column-wise into $\Theta \in \mathbb{R}^{n_\theta \times B}$ and calling Algorithm 2 solves the entire batch simultaneously, e.g., to evaluate a planner across a distribution of scenarios [2] rather than one scenario at a time.

A. A device- and strategy-agnostic solver loop

For convenience, we stack the batch's parameter vectors column-wise into $\Theta \in \mathbb{R}^{n_\theta \times B}$, so that column b is instance b's parameter vector θ_b; likewise, we stack each instance's iterate into $X, \Lambda, M, Z \in \mathbb{R}^{(\cdot)\times B}$. Algorithm 1 is then implemented *once*, against a small set of verbs—`residual!`, `jacobian!`, `factorize!`, `ldiv!`—that consume and return plain $(\cdot) \times B$ arrays. The numeric representation of the batched Jacobian ∇K_ρ and its factorization never leave these four verbs: they are encapsulated inside a strategy-specific cache, so the solver's control flow is agnostic to both *where* it runs and *how* ∇K_ρ is represented and factored. We implement a "batched sparse" strategy: the B instances share one sparsity pattern for ∇K_ρ, computed once from the symbolic (G, H), and each instance fills only its own column of the corresponding $(\mathrm{nnz} \times B)$ value array containing only the structurally nonzero elements of ∇K_ρ.

B. Adapting Algorithm 1 to a batch

Three changes distinguish the batched loop in Algorithm 2 from Algorithm 1. First, the homotopy and regularization parameters ρ, ϵ, and the convergence check $\|K_\rho\|$, all become length-B vectors, one per instance, so that harder problem instances can iterate longer without penalizing easier instances in the same batch.

Second, because linesearch (9) has a closed-form solution, the batched stepsizes $\alpha_k, \beta_k \in \mathbb{R}^B$ are therefore computed elementwise. There is no backtracking loop and, in particular, no per-instance synchronization between host and device.

Third, a batch can often include easy, hard, and infeasible instances. Therefore, after an instance exceeds a maximum number (e.g., 5) of consecutive outer iterations without satisfying $\|K_\rho\| \le \rho$, it is flagged as a failure. Depending upon the hardware backend (CPU or GPU), these failures are either skipped or retained, but kept frozen. This automatic detection of failures prevents a handful of stalled instances from slowing down a batch composed of many easier instances.

Stalled instances are not left to run: line 6 restricts the (re)assembly, factorization, and solve to active instances. This prevents a handful of stalled or failed instances from consuming Newton iterations once flagged; otherwise, they would force the entire batch to consume the maximum iteration count.

C. Backends

On CPU, the shared sparsity pattern is assembled once and each instance's numeric factorization of ∇K_ρ reuses the pivot ordering computed on the first outer round; the B instances are distributed across threads. Because each instance's factorization is an independent per-instance sparse LU (KLU) [7], skipping an inactive instance at line 6 is a literal per-instance branch: an instance with $\text{status}_b \neq$ `active` costs nothing beyond a boolean check, so CPU throughput scales with the number of instances still active, not B.

The GPU backend cannot make the same trade. It factors and solves the batch's shared sparsity pattern with `cuDSS`'s *batched* sparse solver, which processes all B instances as a single device-side call and there is no per-instance skip inside a batched factorization. Consequently, line 6's active-instance restriction is accepted on GPU only to maintain a uniform verb signature across backends, but is a no-op there: every outer round factors and solves *all* B instances, converged and failed ones included, and only their zero stepsize keeps the frozen instances' $(x_b, \lambda_b, \mu_b, z_b)$ unchanged. This imposes a real cost on GPU—one paid in exchange for offloading the whole batch to a single, highly parallel factorization—and it is the reason T_{stall} matters more on GPU than on CPU.

Algorithm 2: Batched primal-dual interior point

Input: batch of B instances $(x_b, \lambda_b, \mu_b, z_b; \theta_b)$ as in Algorithm 1, each with its own ρ_b, ϵ_b; tightening/loosening rates $\underline{\gamma}, \bar{\gamma}$; max. inner/outer iterations $k_{\max}, T_{\max}$; max. consecutive stalled outer rounds T_{stall}; tolerance $\bar{K}$

Output: $\text{status}_b \in \{\texttt{solved}, \texttt{failed}\}$ and $(x_b, \lambda_b, \mu_b, z_b)$ for $b = 1, \ldots, B$

1 $\text{status}_b \leftarrow$ `active`, $\text{stall}_b \leftarrow 0 \quad \forall b$
2 $t \leftarrow 0$
3 **while** $t < T_{\max}$ *and* $\text{status}_b =$ `active` *for some* b **do**
4 $k \leftarrow 0$
5 **repeat**
6 assemble and factor ∇K_{ρ_b} and solve for δ_b, for every b with $\text{status}_b =$ `active`
7 step active instances via fraction-to-boundary (9); instances with $\text{status}_b \neq$ `active` take a zero step
8 $k \leftarrow k + 1$
9 **until** $\|K_{\rho_b}\| \le \rho_b$ *for every active* b, *or* $k \ge k_{\max}$
10 **foreach** b *with* $\text{status}_b =$ `active` **do**
11 **if** $\|K_{\rho_b}\| \le \rho_b$ **then**
12 **if** $\rho_b \le \bar{K}$ **then**
13 $\text{status}_b \leftarrow$ `solved`
14 **else**
15 tighten ρ_b, ϵ_b (as in Algorithm 1); $\text{stall}_b \leftarrow 0$
16 **else**
17 loosen ρ_b, ϵ_b; $\text{stall}_b \leftarrow \text{stall}_b + 1$
18 **if** $\text{stall}_b \ge T_{\text{stall}}$ **then** $\text{status}_b \leftarrow$ `failed`
19 $t \leftarrow t + 1$
20 **return** $\{\text{status}_b, (x_b, \lambda_b, \mu_b, z_b)\}_{b=1}^B$

Remark 1. *The verb set of Section IV-A is deliberately narrow in order to readily accommodate other strategies tailored to different problem structures. For example, a "batched dense" strategy could represent ∇K_ρ densely, as a $(\cdot)\times(\cdot)\times B$ tensor, and solve it with a batched LU factorization.*

V. Experiments

We evaluate `MixedComplementarityProblems.jl` against `PATH` along the two axes that matter for the robotics

applications of Section I: reliability—*does the solver converge as often as* PATH*?*—and throughput—*how quickly can it clear a batch of parametrically-related instances?* Concretely, our experiments answer four questions:

1) **Single-instance parity.** Solving one problem at a time, does our interior point method match PATH's reliability, and how does its per-solve wall-clock compare? (Section V-B)
2) **Batched throughput.** When a whole batch must be cleared, how much faster is a single batched solve than sequential PATH calls? (Section V-C)
3) **CPU thread scaling.** How does batched throughput scale with the number of CPU threads? (Section V-D)
4) **GPU vs. CPU.** When does offloading the batch to a GPU beat a many-threaded CPU, and why? (Section V-E)

A. Experimental setup

a) Problem families: We benchmark on two families of parametric MCPs. The first is a family of randomly generated sparse, convex *quadratic programs* (QPs) of the form $\min_x \frac{1}{2}x^\top Mx - \phi^\top x$ s.t. $Ax - b \geq 0$, with $M \succeq 0$; the parameter vector $\theta = (M, A, b, \phi)$ is drawn at random with a controllable sparsity rate, and the problem dimensions (numbers of primal variables n_x and inequality constraints n_μ) are free hyperparameters. This family exercises the solver on a controlled range of KKT system sizes and, because the random instances are not guaranteed to be feasible, on a realistic mix of solvable and infeasible instances. The second family is the lane-change *trajectory game* of Section II-C (the running example), a two-player generalized Nash equilibrium problem over a planning horizon $T \in \mathbb{N}$; each sampled instance randomizes the players' initial states and lane preferences, so the batch spans a distribution of scenarios exactly as a scenario-based planner would encounter [2]. The horizon T controls the per-instance KKT dimension.

b) Baselines: The primary baseline is PATH [1], accessed through PATHSolver.jl [8] and ParametricMCPs.jl [9]; PATH solves one instance at a time on a single thread. We additionally report our own *unbatched* interior point solver (Algorithm 1) as an intermediate baseline, isolating the performance of the interior point formulation from the batching machinery. Against these, we compare the batched interior point solver (Algorithm 2) in both its CPU (multithreaded) and GPU (cuDSS) configurations.

c) Hardware and software: All experiments run on a single workstation with an AMD Ryzen 9 7950X CPU (16 physical cores / 32 hardware threads) and 126 GB of RAM; GPU experiments use an NVIDIA GeForce RTX 4090 (24 GB). We use Julia 1.12.6, run the CPU-multithreaded solver with 32 threads, and access PATH through PATHSolver.jl. All timings are taken after a separate warm-up solve to exclude compilation. Per-solve times (Table I) are summarized as mean $\pm$ standard deviation over the B instances of a batch; batched throughput (Table II) is the total wall-clock to clear the batch, which varies modestly across repetitions (standard deviation under about 20%). Unless noted, we use a convergence tolerance of $\bar{K} = 10^{-4}$ and report each batch's *solved fraction* (instances reaching $\|K_0\| \leq \bar{K}$) and its total wall-clock time to clear all B instances.

Problem	Solver	Solved	Time (ms)
Random QP	PATH	437/1024	4.4 ± 5.1
	Algorithm 1 + UMFPACK	431/1024	17.7 ± 19.6
	Algorithm 1 + KLU	431/1024	4.7 ± 6.3
Trajectory game	PATH	996/1024	45.2 ± 22.3
	Algorithm 1 + UMFPACK	1024/1024	5.5 ± 4.4
	Algorithm 1 + KLU	1024/1024	0.9 ± 0.5

TABLE I: Single-instance reliability and speed: solved fraction and per-solve wall-clock (mean $\pm$ std) over 1024 random instances of each family (QP: $n_x = 32$, $n_\mu = 16$; trajectory game: $T = 10$). The infeasible QP instances make the per-solve distribution bimodal, so the mean sits well above the median (QP medians: PATH 4.1, Algorithm 1 + UMFPACK 2.5, Algorithm 1 + KLU 0.6 ms).

B. Single-instance reliability and speed

Before batching, we evaluate Algorithm 1 against PATH on individual (unbatched) problems. Table I reports, for each problem family, the fraction of instances each solver converges on and its per-solve wall-clock, over 1024 random instances (QP: $n_x = 32$, $n_\mu = 16$; trajectory game: $T = 10$). Two results stand out. First, Algorithm 1 is as reliable as PATH: it converges on 431 of the 1024 QP instances to PATH's 437 (the remainder being infeasible), and on *all* 1024 trajectory games, where PATH declares 28 failures. Second, in contrast to what one might expect against a mature, heavily optimized solver, once its Newton system is factored with KLU (see below), Algorithm 1 is per-solve *competitive with, and often faster than*, PATH: its median solve is roughly $7\times$ faster on the QP and $55\times$ faster on the trajectory game. On the QP its *mean* is nonetheless comparable to PATH's, because the infeasible random instances form a heavy tail—they iterate to the round limit before being declared failures—which inflates the mean well above the median (hence the large standard deviations in Table I). The batched results below then add a further, orthogonal speedup from parallelism.

The sparse linear solver used inside Algorithm 1 matters substantially here. Algorithm 1 refactorizes the regularized Jacobian of (8) at every Newton step, always with the same sparsity pattern. KLU [7] and UMFPACK [10] are both standard direct solvers for sparse linear systems; KLU's in-place refactorization, which reuses the symbolic analysis and pivot ordering across steps, is 4–6$\times$ faster per solve than UMFPACK on these repeatedly refactored systems, at identical reliability (Table I). We therefore adopt KLU by default; it is also the factorization the batched CPU backend uses (Section IV).

C. Batched CPU throughput vs. PATH

The batched solver's purpose is to clear an entire batch at once. Table II reports the total wall-clock to solve a batch of

Problem	Solver	Total time	Solved	Speedup
Random QP	PATH	3.84 s	437/1024	1×
	Algorithm 1	5.61 s	431/1024	0.7×
	Algorithm 2	0.58 s	424/1024	**6.6**×
Trajectory game	PATH	46.5 s	996/1024	1×
	Algorithm 1	0.97 s	1024/1024	47.9×
	Algorithm 2	0.44 s	1024/1024	**105.7**×

TABLE II: Batched throughput on CPU. Total wall-clock time and solved fraction for clearing a batch of $B = 1024$ instances, and speedup over sequential PATH. Both our solvers use the KLU factorization in solving (8).

$B = 1024$ instances of each family three ways: sequential PATH calls, sequential unbatched Algorithm 1 calls, and via a single Algorithm 2 call across 32 CPU threads. As previewed in Figure 1(b), on the trajectory game the CPU-multithreaded batched solver clears the batch roughly $110\times$ faster than sequential PATH; even the *unbatched* solver is already $\sim 48\times$ faster than PATH here, because its per-solve cost is so much lower (Table I), and batching then adds a further $\sim 2\times$ on top.

On the QP the batched solver is $6.6\times$ faster than PATH, even though the *sequential* unbatched solver is here slightly slower than PATH in total (Table II). This is a consequence of the infeasible instances. Roughly half of the random QPs are infeasible, and each such instance runs all the way to Algorithm 1's outer-iteration limit before it is declared a failure; these few expensive solves dominate the *sequential* total, even though the typical (feasible) solve is fast (Table I). The batched solver absorbs this tail: the infeasible instances are factored and stepped in parallel with the rest, and the per-instance stall detection of Algorithm 2 freezes them early, so they no longer gate the total. The batched solver therefore clears the whole batch $6.6\times$ faster than PATH despite the tail.

The batched speedup comes from two compounding sources: sharing the symbolic (G, H) structure across the batch (assembled and sparsity-analyzed once), and parallelizing the per-instance factorize/solve across threads. Comparing against the sequential Algorithm 1 isolates the second effect—due purely to parallelization—from the first: it is $9.7\times$ faster on the QP but only $2.2\times$ faster on the game, where each per-instance solve is already inexpensive, so there is less serial work for threading to absorb.

D. CPU thread scaling

Figure 2 shows batched throughput (instances cleared per second) as a function of the number of CPU threads, for a fixed batch size. Because each instance's sparse factorization is independent (cf. Section IV), throughput scales **[TODO: near-linearly? sublinearly?]** up to the number of physical cores and then **[TODO: plateaus/regresses]** when the physical cores are saturated. **[TODO: report the peak throughput and the thread count at which it is reached, for both problem families]**

[TODO: thread-scaling plot: throughput (instances/s) vs. thread count, one curve per problem family, with a vertical marker at the physical core count]

Fig. 2: Batched-solver throughput vs. CPU thread count at fixed batch size. Throughput scales with the number of active instances until the physical cores are saturated.

E. GPU vs. CPU: a regime-dependent crossover

Whether the GPU backend beats the multithreaded CPU depends on the problem regime. Because both backends run the same solver loop behind the verb set of Section IV, this comparison isolates the linear-algebra backend. We sweep two axes independently (Figure 3): the batch size B at fixed per-instance size (left), and the per-instance size at fixed B (right)—controlled by the number of primal variables n_x in the QP and by the horizon T in the trajectory game.

GPU outperforms batched CPU only once each per-instance KKT system is both large *and* dense. On the QP, the GPU backend pulls ahead as the batch grows larger (up to $1.8\times$ faster than the 32-thread CPU at $B = 4096$) and, more strongly, as the per-instance system grows: at $n_x = 128$ the GPU is 2–3× faster. On the trajectory game, by contrast, the CPU is faster at *every* planning horizon we tested: at $T = 30$ ($d \approx 2000$) the GPU takes 7.9 s to the CPU's 4.5 s ($1.7\times$ slower), and at $T = 50$ ($d \approx 3500$), 17.9 s to 12.6 s. Both backends nonetheless clear these batches far faster than sequential PATH.

On its face, these results appear counterintuitive: the trajectory game KKT systems are the *largest* we solve, yet here the GPU backend is uniformly slower than batched CPU. Two opposing effects govern end-to-end cost, and they separate cleanly when the batched linear-algebra verbs (`jacobian!`, `factorize!`, `ldiv!`) are timed in isolation. *Per Newton step, with the whole batch active*, the GPU's batched sparse factorization does become cheaper than the CPU's per-thread KLU as each instance's Jacobian grows: its combined assemble-and-factorize cost crosses over the CPU's at $d \approx 2500$ ($T \approx 35$) and is $\sim 1.4\times$ cheaper by $d \approx 4900$, because cuDSS amortizes its fixed launch and occupancy overhead better as the matrix grows while the CPU factorization has no comparable fixed cost. *However, a full solve does not run with the whole batch active.* Algorithm 2's set of active instances shrinks as instances converge or are frozen, and on the CPU each step factorizes only the still-active instances (Section IV), whereas on GPU cuDSS always processes the entire batch.

[TODO: two-panel GPU-vs-CPU plot from `benchmark/results/` (`experiments.csv` gpu_scaling/problem_size rows, `per_rep.csv`): (left) CPU/GPU throughput ratio vs. batch size B at fixed per-instance size; (right) vs. per-instance size (QP n_x / game horizon T) at fixed B. A curve per family (QP, game); horizontal line at $1\times$ marks parity.]

Fig. 3: GPU (`cuDSS`) vs. CPU batched throughput. The GPU pulls ahead only for large, dense per-instance systems (the QP at large n_x or large B); on the sparse trajectory-game systems the multithreaded CPU is faster at every horizon, because its active-set skip factorizes only the still-active instances each step while `cuDSS` always processes the whole batch.

Measured at $d = 3500$, the GPU's per-step advantage of $1.3\times$ with all 1024 instances active inverts to an $8\times$ *disadvantage* once only 32 remain active. Since a real solve spends most of its steps with a small group of active instances, the CPU wins end-to-end. The GPU therefore prevails only when each per-instance system is large enough for `cuDSS` to win the per-step comparison *and* dense enough—as in the large-n_x QP—that its factorization advantage is large; the trajectory game KKT systems meet the first condition but not the second.

Two further caveats bear mention. First, the fraction of solved instances in a batch of trajectory games declines at longer time horizons (for our specific test problem), even with warm starting (92% at $T = 30$, 57% at $T = 50$), and cold starting collapses it further, so we restrict the game comparison to the warm-started regime and caution against over-interpretation of results on very long horizons. Second, the fraction of solved instances in a batch of QPs changes sharply with n_x ($41\% \to 96\% \to 100\%$ from 32 to 128 primals), which complicates the interpretation of those timing results.

VI. Discussion and Future Work

[TODO: TODO]

References

[1] S. P. Dirkse and M. C. Ferris, "The path solver: a nommonotone stabilization scheme for mixed complementarity problems," *Optimization methods and software*, vol. 5, no. 2, pp. 123–156, 1995.

[2] J. Li, C. Y. Chiu, L. Peters, F. Palafox, M. Karabag, J. Alonso-Mora, S. Sojoudi, C. Tomlin, and D. Fridovich-Keil, "Scenario-Game ADMM: A parallelized scenario-based solver for stochastic noncooperative games," in *62nd IEEE Conference on Decision and Control, CDC 2023*. IEEE, 2023, pp. 8093–8099.

[3] L. Peters, A. Bajcsy, C.-Y. Chiu, D. Fridovich-Keil, F. Laine, L. Ferranti, and J. Alonso-Mora, "Contingency games for multi-agent interaction," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2208–2215, 2024.

[4] B. Amos and J. Z. Kolter, "OptNet: differentiable optimization as a layer in neural networks," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, 2017, pp. 136–145.

[5] F. Facchinei and J.-S. Pang, *Finite-dimensional variational inequalities and complementarity problems*. Springer, 2003.

[6] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.

[7] T. A. Davis and E. Palamadai Natarajan, "Algorithm 907: Klu, a direct sparse solver for circuit simulation problems," *ACM Transactions on Mathematical Software (TOMS)*, vol. 37, no. 3, pp. 1–17, 2010.

[8] C. Kwon, O. Dowson, *et al.*, "PATHSolver.jl," 2017. [Online]. Available: https://github.com/chkwon/PATHSolver.jl

[9] L. Peters, "ParametricMCPs.jl," 2023. [Online]. Available: https://github.com/JuliaGameTheoreticPlanning/ParametricMCPs.jl

[10] T. A. Davis, "Algorithm 832: Umfpack v4. 3—an unsymmetric-pattern multifrontal method," *ACM Transactions on Mathematical Software (TOMS)*, vol. 30, no. 2, pp. 196–199, 2004.